■ Detailed Documentation: Level Score Calculation System

This document provides a line-by-line explanation of the Level Score Calculation system. The system determines user scores based on performance, fairness rules, penalties, and bonuses. It applies to drivers, delivery partners, and merchants with tier mapping into Bronze, Amber, Ruby, and Gold.

1. Percentile Rank Function

This function calculates where a user's value stands relative to the population.

```
def percentile_rank(value, population):
    less = sum(1 for x in population if x < value)
    equal = sum(1 for x in population if x == value)
    return (less + 0.5 * equal) / len(population) if population else 0
```

2. Streak Score

This rewards login streaks using exponential growth. It rises quickly at first, then slows down as days increase.

```
def streak_score(streak_days, tau):
    return 1 - math.exp(-streak_days / tau)
```

3. Tier Mapping

Maps scores to tier categories Bronze, Amber, Ruby, and Gold.

```
def get_tier(score):
    if score <= 250: return "Bronze"
    elif score <= 500: return "Amber"
    elif score <= 750: return "Ruby"
    else: return "Gold"
```

4. Fairness Application

This function applies penalties and consistency bonuses. Inactivity penalties scale by days, inconsistency adds flat penalty, and fairness rules protect lower-tier users.

```
def apply_fairness(score, tier, inactivity_days, inconsistent_days, was_active=True):
    penalty = 0
    if inactivity_days > 0:
        penalty += int(100 * (1 - math.exp(-inactivity_days / 30)))
    if inconsistent_days > 0:
        penalty += 30

    tier_penalty_factor = {
        "Bronze": 0.5,
        "Amber": 0.75,
        "Ruby": 1.0,
        "Gold": 1.0
    }
    penalty = int(penalty * tier_penalty_factor[tier])

    score_after_penalty = max(0, score - penalty)
    consistency_bonus = 0
    if was_active and inconsistent_days == 0 and inactivity_days == 0:
        consistency_bonus = 20
        score_after_penalty = min(1000, score_after_penalty + consistency_bonus)
    return score_after_penalty, penalty, consistency_bonus
```

5. Role-Specific Raw Score

Each role has different weight distributions for their features. Drivers rely heavily on rides, delivery partners on job acceptance, and merchants on fulfilment and order value.

```
# Example for driver:
R_raw = 0.10*L + 0.10*S + 0.35*V + 0.20*OT + 0.15*CR + 0.10*R
```

6. Monthly Gain

Performance percentile is scaled to produce monthly score gain, capped to avoid unfair jumps.

```
R_pct = percentile_rank(R_raw, population_samples["R_raw_values"])
monthly_gain = 1000 * R_pct * growth_rate
monthly_gain = min(monthly_gain, max_monthly_gain)
```

7. Score Progression

Adds monthly gain to the previous score to get the initial score, then maps to a tier.

```
initial_score = prev_level_score + monthly_gain
initial_tier = get_tier(initial_score)
```

8. Apply Fairness in Update

Fairness penalties and consistency bonuses are applied at this stage to adjust the score.

```
score_after_penalty, penalty, consistency_bonus = apply_fairness(
    initial_score, initial_tier, inactivity_days, inconsistent_days, was_active=True
)
```

9. One-Time Boost

If the user is new but has past company experience, they get a one-time boost (scaled).

```
if months_active == 1 and first_time_account and worked_in_company_before:
    raw_boost = 100
    boost_applied = raw_boost * weight_boost
```

10. Final Score & Tier

Final score is capped at 1000, mapped to the final tier, and includes all details (penalty, bonus, boost).

```
final_score = min(1000, score_after_penalty + boost_applied)
final_tier = get_tier(final_score)
```